

第064讲 最大间隔与软间隔实验指南

1 你要核验的是同一个优化问题

实验将线性SVC与一个使用显式松弛变量的SciPy原始求解器对照。你需要从手算四点问题开始，确认函数间隔、几何距离和hinge的区别，再观察C、异常点与特征单位如何影响真实边界。正式模型选择只用validation，test封存到选择完成。

随包提供完整数据、生成器、实验代码、六张图、已执行Notebook和普通/-O测试。数据为原创非个人合成数据，CPU即可运行，安装完成后不联网。最后交付自己的运行报告、检查结果与一页解释，而不只交一个准确率。

2 环境与目录

在终端进入包含experiment.py的第064讲目录，按以下方式创建环境：

```
conda env create -f environment.yml
conda activate ml064
python --version
python -c "import sklearn, scipy; print(sklearn.__version__, scipy.__version__)"
```

本讲验证版本为Python3.12.14、scikit-learn1.8.0、SciPy1.17.0。已有Python3.12隔离环境可按requirements.txt安装。首次环境创建需要从官方来源联网获取依赖，数值实验无需网络。不要在没有记录的情况下切换到另一个SVM类，因为默认损失与截距惩罚可能不同。

PDF阅读不需要额外软件。源码重建使用Markdown、WeasyPrint、MathJax3.2.2、Pango与Noto CJK字体，依赖列于build-requirements.txt和package.json。bash build_all.sh在已有环境中运行，不自动安装，并把重建结果放到outputs而非覆盖冻结文件。

3 数据与实验协议

train.csv有64行，validation.csv有80行，test.csv有160行。id是唯一标识，不参与预测；x1与x2是两个连续特征；y只取负1与正1。生成器先按两类构造不同x1中心的Gaussian点，x2提供独立变化，再把训练第0行固定成x=(1.8,0.8)、y=-1，作为预先指定的异常点。

三个角色独立生成且ID不交叉。测试数据没有人为加入这一个指定异常点，但各类Gaussian分布本身仍有重叠。这个设计用于几何教学，不代表现实异常率与部署效果。CSV有十位小数的明确输出格式，生成器与摘要都随包保存。

```
python generate_data.py --directory outputs/regenerated-data
```

该命令把重现数据写到新目录。common.load_data会同时核对SHA256、原生成器字节和ID互斥性。不要关闭检查来适配随手改过的CSV。探索新数据时应复制实验并重新冻结协议。

4 手算四点问题

在纸上画出负类(-1,-1)、(-1,1)与正类(1,-1)、(1,1)。先猜边界，再证明任何可行解都有 $w_1 \geq 1$ ，因此最小范数平方至少为1。 $w=(1,0)$ 、 $b=0$ 恰达到下界，目标为0.5，半宽1、总宽2。

```
from margin import geometry, objective, solve_primal
X = [[-1,-1],[-1,1],[1,-1],[1,1]]
y = [-1,-1,1,1]
print(geometry(X, y, [1,0], 0))
print(objective(X, y, [1,0], 0, 1))
print(solve_primal(X, y, 100))
```

把 w 改成 $[3,0]$ 再次调用`geometry`。函数间隔应变成3，带标签的真实距离仍是1。注意`canonical_full_width`字段此时变成2/3：它表示 f 等于正负1的两条数值等高线宽度，不再等于最近训练点所定义的最大间隔。这一小实验是防止记号混淆的关键。

5 运行四组C与独立原始求解

```
python experiment.py --out outputs/my-run/result.json
```

程序训练 $C=0.03$ 、 0.3 、 3 、 30 四个线性核SVC，逐个保存 w 、 b 、支持向量索引、hinge松弛、验证准确率与fit时间。预先规定准确率并列时选列表最前者，因此本次选择索引0，即 $C=0.03$ 。

随后使用相同训练数据与 C 调用`solve_primal`。请定位代码中的 $z=(w,b,x_i)$ 、目标梯度以及矩阵 A ，每行约束都是 $y_i f(x_i) + x_i \geq 1$ 。程序的可行初值为 $w=0$ 、 $b=0$ 、所有 $x_i=1$ 。

冻结结果的两个求解器目标差约 $1.67e-10$ ，训练分数最大差约 $1.09e-8$ 。目标匹配比参数逐位完全相等更有意义：数值求解有容差，某些问题还可能存在非唯一偏置或乘子。当前例子同时获得很小的分数差，但不要把这种强匹配当作每个退化问题的必然要求。

6 阅读三种诊断而不改正式选择

```
python plots.py --report outputs/my-run/result.json --directory outputs/my-run/figures
```

第一张图展示函数间隔与几何距离的尺度差异；第二张图对照hinge和0-1错误；第三、四张图展示 C 与边界、带宽、hinge总量的关系。第五张图保持选定 C ，移除预指定异常训练行再画边界。第六张图把 x_2 乘100，比较不缩放和训练集标准化。

这些诊断不参与正式模型选择。异常点诊断不是数据清洗授权；单位诊断也不能据此重新比较测试效果。缩放图把两条边界都转换回原坐标绘制，确保不是看到了坐标单位改变制造的错觉。

本次四组验证准确率依次为0.9375、0.925、0.925、0.9375。封存测试准确率0.95。单位诊断在验证集得到0.9125与0.9375，仅作为预设诊断观察。所有这些数字都来自实际执行，单次结果不等于统计显著性结论。

7 普通与优化模式测试

```
python test_experiment.py --report outputs/my-run/result.json --out outputs/my-run/
tests.json
python -O test_experiment.py --report outputs/my-run/result.json --out outputs/my-
run/tests-optimized.json
```

两者应均status=passed。检查覆盖解析四点解、几何尺度不变性、hinge各区间、原始可行性、库目标匹配、C的训练目标权衡、测试指标重算、非法输入与完整新运行。还会故意同时改CSV和摘要、以及篡改报告，确认程序拒绝伪造的一致性。

不要只看checks数量。找出一项测试，自己解释它排除了什么错误、还排除不了什么错误。例如“原始约束可行”排除了违反约束的结果，却单独不能证明目标最优；本讲还需手算和独立库对照，下一讲进一步用对偶间隙检验。

8 Notebook重现

```
python execute_notebook.py --out outputs/my-run/experiment.ipynb
```

执行器在新Python进程内启动真实IPython InProcessKernel，运行手算、主实验、绘图和测试，并把六张图嵌入输出。浏览器Jupyter界面与外部套接字内核传输不属于本次验证。打开结果Notebook确认每格有执行编号、无error输出、图像本身保存而不只是外部文件链接。

若机器较慢，SLSQP求解和测试重跑需要更多时间；不要截断运行却声称通过。先看求解器success及约束残差，再检查版本与数据。若输入出现NaN、标签不是负1/正1、C不是正有限数，本讲教学求解器会显式拒绝。

9 你的实验报告应回答什么

解释硬间隔为什么会不可行，软间隔如何用代价允许违反；用一个样本证明分对仍可能有hinge损失；说明C越大不等于相对正则越强。列出训练、验证、测试以及诊断的各自用途。

保留完整运行报告与六图，说明目标差和约束残差的量级。记录耗时的测量范围，不把本机秒数当成理论复杂度。最后写一条限于本数据与此协议的结论，以及一个需要新验证资源的后续问题。

如果你能指出为什么间隔不是概率、为什么特征标准化必须只在训练集fit，以及为什么删掉一个错例不等于完成科学清洗，就完成了本讲的主要验收。